The Web Application Hacker's Handbook

Trouver et exploiter les failles de sécurité (2e édition)

de Dafydd Stuttard et Marcus Pinto (Wiley, 2011)

Résumé complet du livre

Aperçu chapitre par chapitre avec des outils, des techniques et des points clés à retenir

Préparé comme compagnon de lecture – et non comme substitut au texte intégral.

À propos de ce résumé

Ce document résume les 21 chapitres du Web Application Hacker's Handbook, 2e édition de Dafydd Stuttard et Marcus Pinto, largement considéré comme la référence encyclopédique définitive sur les tests d'intrusion des applications Web. Contrairement à une procédure pas à pas menée en laboratoire, ce livre est organisé comme un catalogue approfondi, vulnérabilité par vulnérabilité : pour chaque catégorie de faille, il couvre le mécanisme sous-jacent, comment le trouver, comment l'exploiter et comment y remédier, en terminant par une méthodologie de liste de contrôle principale (Chapitre 21).

Chaque section de chapitre ci-dessous comprend un synopsis, des techniques/concepts clés et un encadré répertoriant les outils auxquels les auteurs font référence pour ce sujet. Parce que ce livre met l'accent sur la méthodologie et la technique manuelle plutôt que sur les outils automatisés, beaucoup moins d'outils nommés apparaissent que dans un pur manuel de laboratoire – les auteurs se concentrent explicitement sur une petite boîte à outils éprouvée (centrée sur leur propre outil, Burp Suite) plutôt que sur une visite exhaustive des outils. Les outils apparaissant pour la première fois sont marqués en orange.

Structure du livre (4 sections logiques)

- Définition du contexte (Ch. 1 à 4) : l'état de la sécurité des applications Web, les mécanismes de défense de base, l'introduction aux technologies Web et la cartographie de la surface d'attaque d'une application.
- Attaquer les mécanismes de défense de base (Ch. 5 à 8) : contrôles côté client, authentification, gestion de session et contrôle d'accès.
- Attaquer des classes de vulnérabilité spécifiques (Ch. 9 à 18) : magasins de données (injection SQL/NoSQL/XPath/LDAP), composants back-end, logique d'application, XSS, autres attaques côté utilisateur, automatisation, divulgation d'informations, code natif, architecture d'application et serveur d'applications.
- Rassembler les choses (Ch. 19 à 21) : examen du code source, boîte à outils du pirate informatique et méthodologie/liste de contrôle d'attaque complète.

Chapitre 1 : (In)sécurité des applications Web

Préparez le terrain : malgré les assurances répandues selon lesquelles « ce site est sécurisé », la plupart des applications Web restent exploitables avec des compétences modestes. Le chapitre décrit la cause première de presque toutes les vulnérabilités Web : les applications doivent accepter des entrées arbitraires et non fiables de la part des utilisateurs, et des défauts surviennent chaque fois que ces entrées sont traitées de manière non sécurisée.

Cela explique également pourquoi les failles des applications Web sont importantes au-delà de l'application elle-même : une application Web compromise permet souvent à un attaquant de prendre pied dans le réseau interne plus large d'une organisation, faisant de la couche d'application Web le « périmètre de sécurité » moderne.

Techniques et concepts clés

- Problème principal : « les utilisateurs peuvent soumettre des entrées arbitraires » – toutes les autres classes de vulnérabilité du livre sont une variante de la gestion dangereuse de ces entrées.
- Les applications Web constituent désormais la principale surface d’attaque de la plupart des organisations, bien plus que les périmètres réseau traditionnels.
- Présente le double objectif du livre : attaquer les défauts ET comprendre les conseils de remédiation.

Chapitre 2 : Mécanismes de défense de base

Décrit les mécanismes de sécurité que chaque application doit implémenter pour gérer le fait que les entrées ne sont pas fiables : gestion de l'accès des utilisateurs (authentification, gestion de session, contrôle d'accès), gestion des entrées utilisateur (validation des limites, validation/canonisation en plusieurs étapes) et gestion des attaquants (gestion des erreurs, journalisation d'audit, alerte, réaction aux attaques).

Couvre également les fonctions de gestion des applications (interfaces d'administration) dans le cadre de la surface d'attaque. Comprendre ces mécanismes prévus est une condition préalable pour le reste du livre, puisque chaque chapitre ultérieur cible un mécanisme de défense défaillant spécifique.

Techniques et concepts clés

- Trois piliers du contrôle d'accès des utilisateurs : authentification, gestion de session, contrôle d'accès — chacun recevra son propre chapitre dédié plus tard (6, 7, 8).
- La validation des limites (valider les entrées exactement là où elles traversent une limite de confiance) est présentée comme la stratégie de gestion des entrées la plus robuste par rapport à la mise sur liste noire/à la désinfection seule.
- Des bugs de canonisation surviennent lorsque l'entrée est validée avant d'être décodée/normalisée, laissant passer les attaques codées.

Chapitre 3 : Technologies d'applications Web

Une introduction aux technologies nécessaires pour comprendre le reste du livre : le protocole HTTP (requêtes, réponses, méthodes, en-têtes, cookies, codes d'état), HTTPS, les proxys HTTP et l'authentification, REST, les fonctionnalités côté serveur et côté client (HTML, JavaScript, VBScript et technologies d'extension de navigateur telles que les applets Java, ActiveX, Silverlight, Flash).

Couvre également les concepts d'état/de session et les schémas de codage utilisés dans les applications Web et les attaques : codage URL, codage Unicode, codage HTML, Base64 et codage hexadécimal - qui réapparaissent tous constamment sous forme de techniques d'attaque et d'évasion dans les chapitres suivants.

Techniques et concepts clés

- Les schémas de codage (URL/Unicode/HTML/Base64/hex) sont fondamentaux pour comprendre le contournement des filtres et la livraison des charges utiles dans presque tous les chapitres ultérieurs sur les attaques.
- Distingue la fonctionnalité côté serveur (traitement) de la fonctionnalité côté client (présentation), une distinction centrale dans les attaques de contournement côté client du chapitre 5.
- Les proxys HTTP sont présentés comme la technique de base pour intercepter et manipuler le trafic.

Outils référencés dans ce chapitre

HTTP intercepting proxy (concept)

Chapitre 4 : Cartographie de l'application

Le premier exercice pratique contre n'importe quelle cible : énumération du contenu et des fonctionnalités via une araignée Web automatisée, une araignée dirigée par l'utilisateur et découverte de contenu caché (fichiers de sauvegarde, répertoires, paramètres) grâce à des techniques telles que le forcement brut et l'inférence à partir de conventions de dénomination.

Couvre l'analyse de l'application pour identifier tous les points d'entrée pour les entrées de l'utilisateur, l'identification des technologies et des fonctionnalités côté serveur et la compilation du tout dans une carte de la surface d'attaque complète de l'application avant le début de toute attaque réelle.

Techniques et concepts clés

- Distingue les « pages d'application » des « chemins fonctionnels » — la même fonctionnalité peut être atteinte via plusieurs URL/paramètres qui doivent tous être mappés.
- La découverte de contenu caché (noms de fichiers prévisibles, extensions de sauvegarde, robots.txt, commentaires) révèle souvent des fonctionnalités oubliées, non authentifiées ou de débogage.
- Les empreintes technologiques côté serveur (en-têtes, pages d’erreur, extensions de fichiers) indiquent quelles classes de vulnérabilité sont les plus susceptibles d’être pertinentes ultérieurement.

Outils référencés dans ce chapitre

Web spiders (automated crawling tools)	Intercepting proxy (manual/user-directed spidering)

Chapitre 5 : Contourner les contrôles côté client

Couvre la première véritable catégorie de vulnérabilité : faire confiance au client. Cela inclut les données transmises via le client que le serveur suppose comme non modifiées (champs de formulaire masqués, cookies, paramètres d'URL, en-tête Referer, données opaques/codées et ASP.NET ViewState) et les contrôles côté client sur les entrées de l'utilisateur (limites de longueur, validation basée sur un script, éléments de formulaire désactivés).

Couvre également les technologies côté client plus lourdes : intercepter le trafic depuis et décompiler les extensions de navigateur (applets Java, Flash, Silverlight, ActiveX) et attacher un débogueur pour analyser directement la logique côté client.

Techniques et concepts clés

- Règle d'or : tout contrôle appliqué uniquement au client peut toujours être contourné, car l'attaquant contrôle entièrement son propre navigateur/client.
- ASP.NET ViewState est présenté comme un magasin de données commun côté client, opaque mais décodable.
- La décompilation des extensions de navigateur Java/.NET (par exemple, avec JAD ou un décompilateur/Reflector .NET) peut révéler une logique métier appliquée par le client et des secrets codés en dur.

Outils référencés dans ce chapitre

Intercepting proxy	JAD (Java decompiler)	.NET decompiler / Reflector	Debugger (attaching to client components)

★ Nouveaux dans ce chapitre : JAD decompiler, .NET Reflector

Chapitre 6 : Attaquer l'authentification

Un catalogue complet de failles d'authentification : défauts de conception (mauvais mots de passe, connexion par force brute, messages d'échec verbeux, transmission d'informations d'identification vulnérable, fonctions de changement de mot de passe/mot de passe oublié/"se souvenir de moi"/usurpation d'identité faibles, noms d'utilisateur non uniques ou prévisibles, mots de passe initiaux prévisibles, distribution d'informations d'identification non sécurisée) et défauts de mise en œuvre (logique de connexion en cas d'ouverture, défauts dans les mécanismes de connexion à plusieurs étapes, stockage d'informations d'identification non sécurisé).

Se termine par une section approfondie « Sécurisation de l'authentification » couvrant les informations d'identification solides, la gestion des informations d'identification secrètes, la validation appropriée, la prévention par force brute et la journalisation/alerte - servant à la fois de liste de contrôle pour l'attaquant et de guide de renforcement pour le défenseur.

Techniques et concepts clés

- Les mécanismes de connexion en plusieurs étapes (courants dans les applications bancaires) sont pointés du doigt comme introduisant de nouvelles failles logiques non présentes dans les connexions simples en une seule étape.
- Les messages d'échec/d'erreur verbeux (par exemple, « nom d'utilisateur invalide » ou « mot de passe invalide ») permettent l'énumération du nom d'utilisateur, un précurseur clé du forçage brutal.
- Les fonctions de récupération de compte et de « se souvenir de moi » constituent souvent le maillon le plus faible, contournant une authentification primaire autrement forte.

Outils référencés dans ce chapitre

Intercepting proxy	Burp Intruder (automated brute-forcing)	Hydra

★ Nouveaux dans ce chapitre : Hydra

Chapitre 7 : Attaquer la gestion de session

Les jetons de session permettent à un protocole HTTP sans état de suivre un utilisateur à travers les requêtes – et la rupture de ce mécanisme peut permettre à un attaquant de contourner complètement la connexion. Couvre les faiblesses dans la génération de jetons (jetons significatifs/codés, jetons prévisibles, jetons mal chiffrés) et les faiblesses dans la gestion des jetons (divulgation sur le réseau ou dans les journaux, mappage jeton-session vulnérable, terminaison de session vulnérable, portée libérale des cookies permettant le détournement).

Se termine par la sécurisation de la gestion des sessions : générer des tokens aléatoires forts et les protéger tout au long de leur cycle de vie (attributs sécurisés des cookies, transmission, expiration).

Techniques et concepts clés

- Un mécanisme de gestion de session défectueux peut être aussi grave qu'une authentification brisée, car il permet aux attaquants de se faire passer pour d'autres utilisateurs sans informations d'identification.
- La prévisibilité des jetons peut être analysée statistiquement en échantillonnant de nombreux jetons pour détecter des modèles (compteurs séquentiels, PRNG faibles, jetons basés sur un horodatage).
- La fixation de session et la portée libérale des cookies sont mises en avant comme des vecteurs de détournement subtils mais sérieux.

Outils référencés dans ce chapitre

Intercepting proxy	Burp Sequencer (token randomness analysis)

★ Nouveaux dans ce chapitre : Burp Sequencer

Chapitre 8 : Attaquer les contrôles d'accès

Le contrôle d'accès repose sur le bon fonctionnement de l'authentification et de la gestion des sessions, mais ajoute une troisième couche, à échec indépendant : faire respecter ce qu'un utilisateur authentifié est réellement autorisé à faire. Couvre les vulnérabilités courantes : fonctionnalités totalement non protégées, fonctions basées sur des identifiants (références d'objet directes non sécurisées), fonctions à plusieurs étapes non protégées, fichiers statiques exposés, mauvaise configuration de la plate-forme et méthodes de contrôle d'accès non sécurisées (par exemple, faire confiance à un paramètre caché pour un rôle).

Décrit les tests systématiques : utilisation de différents comptes d'utilisateurs, tests de processus en plusieurs étapes et de scénarios d'accès limité, tests d'accès direct aux méthodes et vérification des restrictions de la méthode HTTP. Se termine par un « modèle de privilèges multicouches » comme défense recommandée.

Techniques et concepts clés

- Insecure Direct Object Reference (IDOR) — manipuler un identifiant (par exemple, un numéro de compte ou de commande) pour accéder aux données d'un autre utilisateur — est un thème central de ce chapitre.

- Les fonctions à plusieurs étapes n'appliquent souvent le contrôle d'accès qu'à la première étape, permettant à un attaquant de passer directement à une étape ultérieure.
- Une défense robuste applique le contrôle d’accès de manière centralisée et cohérente, et non par page/par lien.

Outils référencés dans ce chapitre

Intercepting proxy (multi-account comparison testing)		

Chapitre 9 : Attaquer les magasins de données

Une plongée approfondie dans l'injection SQL : contourner les connexions via l'injection, exploiter les vulnérabilités de base, injecter dans différents types d'instructions SQL, trouver des bogues d'injection, prendre des empreintes digitales dans la base de données, utiliser l'opérateur UNION, extraire des données, contourner les filtres, injection SQL de second ordre et exploitation avancée (canaux hors bande, inférence, techniques de délai/aveugle) - le tout comparé entre MS-SQL, Oracle et MySQL.

Couvre également l'escalade d'une injection SQL vers une compromission complète de la base de données/du système d'exploitation, à l'aide d'outils d'exploitation SQL dédiés (notamment sqlmap), d'une référence rapide sur la syntaxe/erreur SQL et la prévention (requêtes paramétrées). Étend les mêmes concepts d'injection à NoSQL (y compris MongoDB), à l'injection XPath et à l'injection LDAP.

Techniques et concepts clés

- Couvre le spectre complet de l'injection SQL : exfiltration intrabande (basée sur UNION), basée sur les erreurs, aveugle (booléenne/basée sur le temps) et hors bande.
- Les différences spécifiques aux bases de données (syntaxe, messages d'erreur, fonctions/commandes disponibles) sont répertoriées dans MS-SQL, Oracle et MySQL pour une prise d'empreintes digitales pratique.
- sqlmap est présenté comme l'outil automatisé standard pour détecter et exploiter l'injection SQL une fois qu'un paramètre candidat est identifié.
- Les requêtes paramétrées (instructions préparées) sont présentées comme le correctif d'injection SQL définitif.

Outils référencés dans ce chapitre

Intercepting proxy	sqlmap	SQL exploitation tools (generic)

★ Nouveaux dans ce chapitre : sqlmap

Chapitre 10 : Attaquer les composants back-end

Couvre les catégories d'injection au-delà de SQL : injection de commandes du système d'exploitation (via des exemples Perl et ASP), injection dans l'exécution de code dynamique (fonctions de style d'évaluation), manipulation du chemin de fichier (vulnérabilités de traversée du chemin et d'inclusion de fichiers, locales et distantes) et injection dans les interpréteurs XML (y compris l'injection d'entité externe XML/XXE) et les services SOAP.

Couvre également l'injection dans les requêtes HTTP back-end (concepts de falsification de requêtes côté serveur, injection/pollution de paramètres HTTP) et l'injection dans les services de messagerie (manipulation d'en-tête d'e-mail, injection de commandes SMTP) - chacune avec des conseils de détection et de prévention dédiés.

Techniques et concepts clés

- L'injection de commandes du système d'exploitation se cache souvent derrière des fonctions wrapper apparemment sûres qui sont envoyées au système d'exploitation avec une entrée non vérifiée.
- La traversée de chemin ("../") et l'inclusion de fichiers (LFI/RFI) peuvent exposer des fichiers arbitraires ou réaliser l'exécution de code à distance en fonction du langage/de la configuration cible.
- L'injection XXE est signalée comme un vecteur sérieux et souvent négligé pour la divulgation de fichiers et SSRF via des analyseurs XML qui résolvent des entités externes.
- La pollution/injection de paramètres HTTP peut manipuler les requêtes back-end que l'application elle-même adresse à d'autres services.

Outils référencés dans ce chapitre

Intercepting proxy	Manual injection testing

Chapitre 11 : Attaquer la logique des applications

Fait valoir que les failles logiques font partie des vulnérabilités les plus difficiles à détecter automatiquement, car elles sont propres aux règles métier de chaque application, et non à une signature générique comme l'injection SQL. Le chapitre présente 12 études de cas réels de logique brisée — depuis un oracle prévisible de « question de sécurité » jusqu'à une fonction de changement de mot de passe défectueuse, en passant par des flux de travail de caisse/assurance/bancaire manipulables, des abus de remises groupées et des conditions de concurrence dans

logique de connexion.

À partir de ces exemples, les auteurs distillent un ensemble généralisé de tests pour localiser les défauts logiques dans n'importe quelle application : recherchez les hypothèses erronées formulées par les concepteurs sur le comportement des utilisateurs, l'exhaustivité des entrées et le timing.

Techniques et concepts clés

- Les failles logiques peuvent être plus dommageables que les bogues d'injection classiques, car elles ne nécessitent souvent aucune payload particulière : il suffit de faire quelque chose dans un ordre inattendu ou avec des valeurs inattendues.
- Les conditions de concurrence (par exemple, « course contre la connexion ») exploitent l'écart entre un contrôle et l'action correspondante.
- Tester les failles logiques nécessite de comprendre le processus métier, et pas seulement le code/la syntaxe.

Chapitre 12 : Attaquer les utilisateurs : Cross-Site Scripting

Un traitement complet de XSS, décrit comme la vulnérabilité Web la plus répandue. Couvre les trois variétés – XSS réfléchi, stocké et basé sur DOM – avec des exemples concrets et la gamme de charges utiles et de mécanismes de livraison utilisés par les attaquants.

Fournit une méthodologie détaillée pour trouver et exploiter chaque variété XSS, même dans des contextes obscurs ou fortement filtrés, suivie de conseils de prévention (codage de sortie, échappement contextuel, défenses XSS basées sur DOM).

Techniques et concepts clés

- Reflected XSS nécessite un lien/une demande spécialement conçu par victime ; Le XSS stocké persiste dans l'application et frappe automatiquement chaque visiteur suivant, ce qui rend le XSS stocké généralement plus sévère.
- XSS basé sur DOM se produit uniquement dans JavaScript côté client et peut ne jamais toucher le serveur, ce qui nécessite des techniques de détection différentes de celles des charges utiles reflétées par le serveur.
- Le chapitre met l'accent sur le contournement des filtres d'entrée/sortie et des WAF comme une compétence essentielle pour l'exploitation XSS dans le monde réel.

Outils référencés dans ce chapitre

Intercepting proxy	Firebug (DOM/JS inspection)

★ Nouveaux dans ce chapitre : Firebug

Chapitre 13 : Attaquer les utilisateurs : autres techniques

Couvre les attaques contre les utilisateurs au-delà du XSS classique : incitation à des actions de l'utilisateur via la falsification de requêtes (CSRF) et la réparation de l'interface utilisateur (clickjacking), capture de données inter-domaines via l'injection HTML/CSS et le piratage JavaScript, et attaques contre la politique de même origine (y compris avec les extensions de navigateur et HTML5).

Couvre également l'injection d'en-tête HTTP, l'injection de cookies/fixation de session, la redirection ouverte, l'injection SQL côté client, les attaques locales de confidentialité (cookies persistants, contenu mis en cache, historique de navigation, stockage local HTML5), les vulnérabilités du contrôle ActiveX et les attaques purement côté navigateur (enregistrement des frappes, vol d'historique/de recherche, analyse des ports réseau internes, reliure DNS et cadres d'exploitation du navigateur).

Techniques et concepts clés

- CSRF exploite l'inclusion automatique des informations d'identification (cookies) par le navigateur dans les requêtes intersites pour effectuer des actions indésirables en tant que victime.
- La réparation de l'interface utilisateur/le détournement de clics incite les utilisateurs à cliquer sur quelque chose de différent de ce qu'ils perçoivent.
- Les attaques locales contre la confidentialité ciblent les données qu'un navigateur stocke localement (cache, historique, stockage local), quelles que soient les vulnérabilités de chaque application.

- Les cadres d'exploitation de navigateur (par exemple, BeEF) sont référencés comme un moyen de militariser un navigateur accro pour des attaques de type man-in-the-middle contre l'utilisateur.

Outils référencés dans ce chapitre

Intercepting proxy	Browser exploitation framework (e.g., BeEF)

Chapitre 14 : Automatisation des attaques personnalisées

Il affirme que, chaque application étant différente, un piratage Web efficace nécessite une automatisation personnalisée plutôt que des scanners génériques. Couvre l'énumération des identifiants valides, la détection des « hits » dans les réponses groupées, les attaques de script et l'exemple d'outil personnalisé des auteurs (JAttack) pour récolter des données utiles et rechercher les vulnérabilités courantes à grande échelle.

La pièce maîtresse est une présentation détaillée de l'utilisation de Burp Intruder pour automatiser des attaques personnalisées (positions de payload, types de payload, types d'attaque, analyse de réponse), suivie du dépassement des obstacles courants à l'automatisation : mécanismes défensifs de gestion de session et contrôles CAPTCHA.

Techniques et concepts clés

- Les tests manuels de chaque combinaison paramètre/requête ne sont pas évolutifs : une automatisation adaptée à l'application spécifique est essentielle pour une couverture complète.
- Burp Intruder est présenté comme le principal outil permettant de créer et d'exécuter des attaques automatisées personnalisées (force brute, fuzzing, énumération) contre toute requête HTTP.
- Les règles de gestion de session et les stratégies de résolution de CAPTCHA sont couvertes en tant qu'obstacles pratiques que les testeurs doivent contourner pendant l'automatisation.

Outils référencés dans ce chapitre

Burp Intruder	Custom scripts (JAttack example)

★ Nouveaux dans ce chapitre : Burp Intruder

Chapitre 15 : Exploitation de la divulgation d'informations

Couvre comment les applications fuient des informations en cas d'attaque active et comment les testeurs doivent les surveiller en permanence : messages d'erreur de script, traces de pile, messages de débogage informatifs et messages d'erreur détaillés du serveur/de la base de données qui révèlent le fonctionnement interne.

Couvre également l'utilisation d'informations accessibles au public, la conception délibérée de messages d'erreur informatifs, la collecte d'informations publiées sur la cible et l'utilisation de l'inférence pour déduire des détails cachés – en terminant par la prévention :

messages d'erreur génériques, protection des données sensibles, minimisation des fuites côté client.

Techniques et concepts clés

- La divulgation d’informations doit être activement surveillée tout au long de chaque autre phase d’attaque, et non traitée comme un test distinct et isolé.
- Les traces de pile détaillées et les messages d'erreur de base de données révèlent souvent une pile technologique, des chemins de fichiers et même une structure de requête utile pour d'autres attaques (par exemple, injection SQL).
- Déclencher délibérément des erreurs (saisie mal formée, valeurs limites) est une technique permettant d'extraire plus d'informations que ce que le comportement par défaut révèle.

Chapitre 16 : Attaquer les applications compilées natives

Couvre les classes de vulnérabilités de corruption de mémoire qui peuvent survenir dans les composants natifs (C/C++) derrière une application Web : débordements de pile, débordements de tas et vulnérabilités "off-by-one", ainsi que des techniques de détection des vulnérabilités de buffer overflow d'un point de vue Web.

Couvre également les vulnérabilités liées aux entiers (débordements et erreurs de signature) et les vulnérabilités liées aux chaînes de format, en mettant l'accent sur la reconnaissance des symptômes de ces bogues via le comportement des applications Web plutôt que sur le développement complet d'exploits binaires (ce qui, selon les auteurs, est un sujet qui mérite un livre dédié).

Techniques et concepts clés

- Même si la majeure partie du livre concerne les failles logiques/HTTP, les composants de code natifs (par exemple, les filtres ISAPI personnalisés, les bibliothèques de traitement d'images) peuvent introduire des bogues classiques de corruption de mémoire.
- Les dépassements d’entiers et les erreurs de signature peuvent contourner silencieusement les vérifications de longueur/limites.
- Des bogues de formatage de chaîne surviennent lorsque l’entrée de l’utilisateur est transmise directement en tant que spécificateur de format aux fonctions de formatage de style C.

Chapitre 17 : Attaquer l'architecture des applications

Examine les faiblesses au niveau de l'architecture : les architectures à plusieurs niveaux qui ne parviennent pas à séparer correctement les composants, laissant une compromission dans un niveau se transformer en une compromission complète des applications, et les environnements d'hébergement partagé/fournisseur de services d'application (hébergement virtuel, services d'application partagés).

Explique comment attaquer les environnements partagés – dans lesquels une faille dans une application client pourrait compromettre l'hôte sous-jacent ou les applications voisines – et étend cela aux premiers modèles de menace d'hébergement partagé « cloud computing », avec des conseils de sécurisation correspondants.

Techniques et concepts clés

- Une séparation appropriée entre les niveaux d’application limite le rayon d’explosion d’un seul composant compromis.
- L'hébergement partagé/cloud introduit un nouvel acteur menaçant : d'autres locataires sur la même infrastructure, et pas seulement des attaquants externes.
- La sécurisation des environnements partagés nécessite des contrôles d’isolement stricts au niveau de la plateforme, et pas seulement au niveau des applications.

Chapitre 18 : Attaquer le serveur d'applications

Cible le serveur Web/serveur d'applications lui-même plutôt que la logique de l'application : configuration du serveur vulnérable (informations d'identification par défaut, contenu par défaut, listes de répertoires, méthodes WebDAV dangereuses, serveur abusé en tant que proxy ouvert, hébergement virtuel mal configuré).

Couvre également les logiciels serveur vulnérables (failles du framework d'application, vulnérabilités de gestion de la mémoire, bugs d'encodage/canonisation), les techniques de recherche de failles du serveur Web, la sécurisation des logiciels serveur, ainsi que le rôle et les limites des pare-feu d'applications Web (WAF).

Techniques et concepts clés

- Les informations d'identification par défaut et le contenu par défaut/exemple font toujours partie des compromissions les plus courantes sur les serveurs d'applications dans le monde réel.
- Les méthodes WebDAV (PUT, MOVE, etc.) laissées activées peuvent permettre le téléchargement/modification direct de fichiers sur le serveur.
- Les bogues d'encodage/canonisation dans la plate-forme serveur (pas seulement dans l'application) peuvent contourner les restrictions d'accès ou les filtres.
- Les WAF sont présentés comme une couche d'atténuation utile, mais ne remplacent pas la correction des défauts sous-jacents.

Outils référencés dans ce chapitre

Nikto / Wikto (server/CGI vulnerability scanning)

★ Nouveaux dans ce chapitre : Nikto/Wikto

Chapitre 19 : Recherche de vulnérabilités dans le code source

Une approche complètement différente du sondage en boîte noire : examiner le code source réel d'une application. Compare les tests en boîte noire et en boîte blanche et présente une méthodologie de révision du code, puis donne des « signatures » à rechercher/grep dans les classes de vulnérabilité courantes (XSS, injection SQL, traversée de chemin, redirection arbitraire, injection de commandes du système d'exploitation, mots de passe de porte dérobée, bogues logiciels natifs et commentaires de code source suspects).

Fournit des aide-mémoire langue par langue (Java, ASP.NET, PHP, Perl et JavaScript) expliquant comment identifier les données fournies par l'utilisateur, les interactions de session et les API potentiellement dangereuses dans chacune d'entre elles, ainsi qu'une révision du code de la base de données (modèles d'injection SQL, appels de fonctions dangereux) et des outils de navigation dans le code.

Techniques et concepts clés

- L'examen du code source peut détecter des vulnérabilités difficiles ou lentes à détecter en sondant une application en direct, et ne nécessite pas la coopération de la cible si la source est autrement disponible.
- Chaque langage possède des « API dangereuses » caractéristiques (par exemple, eval, exec, concaténation SQL brute) qui servent de points de tri rapides lors d'une révision.
- Une approche de signature basée sur grep permet même aux testeurs ayant une expérience limitée en programmation d'effectuer une révision de code efficace et ciblée.

Outils référencés dans ce chapitre

Code browsing / IDE search tools	grep-style signature searching

Chapitre 20 : Boîte à outils d'un pirate d'application Web

Consolide tous les outils que les auteurs utilisent réellement dans des engagements réels. Couvre les navigateurs Web (Internet Explorer, Firefox, Chrome) et leur rôle dans les tests, puis les suites de tests intégrées - des boîtes à outils centrées sur le proxy d'interception comme Burp Suite - expliquant leur fonctionnement en interne et un flux de travail de test recommandé, ainsi que des alternatives à l'approche du proxy d'interception.

Couvre les scanners de vulnérabilités automatisés autonomes (ce qu'ils détectent, leurs limites inhérentes, les défis techniques auxquels ils sont confrontés et les produits actuellement disponibles à ce moment-là), puis une section sur les « Autres outils » : Wikto/Nikto, Firebug, Hydra et l'écriture de scripts personnalisés — clôturant la discussion sur la boîte à outils pratique du livre.

Techniques et concepts clés

- Burp Suite est la boîte à outils centrale recommandée par les auteurs (proxy d'interception + Répéteur + Intrus + Séquenceur + Scanner, entre autres composants référencés tout au long du livre).
- Les scanners automatisés sont utiles en termes d'étendue et de rapidité, mais ont des limites bien documentées : ils sont aux prises avec des défauts logiques, des processus à plusieurs étapes et des vulnérabilités dépendantes du contexte — les tests manuels restent essentiels.
- Le chapitre indique explicitement qu'aucun outil entièrement automatisé ne peut remplacer des tests manuels qualifiés pour une évaluation approfondie.

Outils référencés dans ce chapitre

Burp Suite (full suite: Proxy, Repeater, Intruder, Sequencer, Scanner) Firebug	Internet Explorer / Firefox / Chrome (with extensions) Hydra	Standalone vulnerability scanners (e.g., commercial DAST products) Custom scripts	Wikto / Nikto

★ Nouveaux dans ce chapitre : Burp Suite (full toolkit)

Chapitre 21 : Méthodologie d'un pirate d'application Web

La pierre angulaire du livre : une liste de contrôle complète et ordonnée qui organise chaque technique des chapitres 1 à 20 en une méthodologie d'attaque unique et reproductible. Structuré autour d'une douzaine de phases principales : cartographie de l'application, test des contrôles côté client, test de l'authentification, test de la gestion des sessions, test des contrôles d'accès, test des vulnérabilités basées sur les entrées, test des vulnérabilités d'entrée spécifiques aux fonctions, test des failles logiques, test des vulnérabilités d'hébergement partagé, test des vulnérabilités du serveur d'applications, vérifications diverses (attaques basées sur DOM, confidentialité locale, force de chiffrement SSL, politique de même origine) et suivi de toute fuite d'informations.

Chaque phase est divisée en étapes numérotées granulaires (par exemple, « 4.3 Test pour l'énumération des noms d'utilisateur », « 7.2 Test pour l'injection SQL ») qui fonctionnent comme une liste de contrôle pratique, à imprimer et à utiliser, pour exécuter un test d'intrusion complet et structuré sur n'importe quelle application Web.

Techniques et concepts clés

- Conçue comme une liste de contrôle autonome et exploitable, les lecteurs peuvent l'utiliser directement lors d'un engagement réel une fois qu'ils ont compris les chapitres sous-jacents.
- Classés par dépendances de tâches logiques (par exemple, comprendre l'application avant de tester la saisie ; tester l'authentification avant les contrôles d'accès).
- Fonctionne comme un résumé pratique du livre – en fait un index condensé de l’ensemble du texte.

Index principal des outils (par première apparition)

Comparé aux livres de style laboratoire, The Web Application Hacker's Handbook garde délibérément son ensemble d'outils nommé petit et profond plutôt que large : les auteurs centrent presque tout autour d'un proxy d'interception (Burp Suite, également créé par Dafydd Stuttard) et d'une technique manuelle, avec une poignée d'outils spécialisés pour des tâches spécifiques. Le tableau ci-dessous répertorie chaque outil nommé, le(s) chapitre(s) où il est couvert et son objectif.

Outil	Chapitre(s)	Objectif
Intercepting proxy (general concept)	3, 4+	Technique de base : intercepter/modifier les requêtes et réponses HTTP(S)
JAD (Java decompiler)	5	Décompile les applets Java pour révéler la logique côté client
.NET Reflector	5	Décompile les assemblys d’extension de navigateur .NET
Hydra	6, 20	Force brute en ligne des formulaires de connexion/identifiants
Burp Sequencer	7	Analyse statistique du caractère aléatoire/prévisibilité du jeton de session
sqlmap	9	Détection et exploitation automatisées de l'injection SQL
Firebug	12, 20	Extension d'outils de développement de navigateur pour inspecter DOM/JavaScript (analyse XSS)
Browser Exploitation Framework (e.g., BeEF)	13	"Hooks" un navigateur victime pour un contrôle continu de style MITM
Burp Intruder	14, 20	Automatise les attaques personnalisées de fuzzing/force brute/énumération à grand volume
Nikto / Wikto	18, 20	Configuration du serveur Web/CGI et analyse des vulnérabilités
Burp Suite (full toolkit: Proxy, Repeater, Intruder, Sequencer, Scanner)	20	La principale suite de tests intégrés des auteurs pour les tests manuels et semi-automatisés
Standalone DAST vulnerability scanners (commercial)	20	Analyse étendue automatisée, avec des limitations notées
Custom scripts (e.g., JAttack)	14, 20	Automatisation sur mesure adaptée aux particularités d'une application spécifique

Notes finales

Ce manuel (2011, 2e édition) est intentionnellement à l'opposé d'un manuel de laboratoire de démarrage rapide : il s'agit d'une référence encyclopédique, vulnérabilité par vulnérabilité, destinée à être lue en profondeur puis utilisée comme référence documentaire continue, la méthodologie du chapitre 21 servant de liste de contrôle pratique et réutilisable. Les auteurs étendent explicitement le livre aux attaques d'applications Web uniquement : les tests de pénétration du réseau, les attaques de pare-feu et la compromission générale des serveurs sont délibérément hors de portée.

Étant donné que le livre est antérieur aux architectures HTML5/SPA/API et aux frameworks modernes largement répandus, certaines références technologiques spécifiques (ActiveX, Silverlight, ASP classique) sont datées. Cependant, les principales classes de vulnérabilités qu'il catalogue – injection SQL/NoSQL/XPath/LDAP, XSS, CSRF, failles d'authentification et de gestion de session, échecs de contrôle d'accès, failles logiques et mauvaise configuration du serveur – restent aujourd'hui l'épine dorsale des tests de sécurité des applications Web, c'est pourquoi ce livre est encore largement considéré comme la référence standard dans le domaine.